

ZENELAIT LMS

COMPREHENSIVE ROLE-BASED LEARNING MANAGEMENT MANUAL

An all-inclusive, multi-role digital learning platform designed for modern educational institutions and training academies. Built by Zenelait InfoTech Private Limited to streamline workflows from classroom attendance to board-level financial and academic audits.

CORPORATE PROFILE

Zenelait InfoTech is a leading software engineering firm dedicated to building next-generation enterprise applications. We specialize in designing customizable SaaS platforms that integrate CRM, ERP, and learning management networks into unified hubs.

ZENELAIT LMS VISION

Zenelait LMS enables educational institutions to achieve 100% operational transparency. Our platform bridges the gap between daily classroom metrics and executive administration auditing, delivering automated compliance checks, integrated payment gateways, and AI-driven student performance insights.

OFFICIAL CONTACT & SECURITY NOTICE: This document is proprietary information of Zenelait InfoTech Private Limited. Unauthorized reproduction or distribution of this manual is strictly prohibited. All system configurations, database connections, and REST API paths are subject to security audits.

TABLE OF CONTENTS

1. System Architecture & Backend Decoupling

2. Role Hierarchies & Dashboard Permissions Mapping

3. Ultra Super Admin Dashboard (System Level Control)

4. Super Admin Portal (Academy Level Management)

5. Admin Portal (Academic Operations & Coordinators)

6. Admin Portal (Timetables, Fees, and Scheduling)

7. Teacher Portal (Classroom logs, Attendance & Grading)

8. Student Portal (Coursework & Certificates Engine)

9. Parent Portal (Auditing & Integrated Razorpay Wallet)

10. AI Assistant Integration (Gemini Contextual Pipeline)

Page 3

Page 4

Page 5

Page 6

Page 7

Page 8

Page 9

Page 10

Page 11

Page 12

Security Note: User role configurations are validated dynamically at database query compile-time. Any route modifications are blocked by client side Auth guards.

1. SYSTEM ARCHITECTURE & BACKEND DECOUPLING

Zenelait LMS employs a highly responsive, decoupled client-server architecture model. The client-side dashboard framework runs on a React Single Page Application (SPA), connecting to a multi-threaded Spring Boot 3 enterprise application that orchestrates business logic and communicates directly with external cloud-based Generative AI models.

TECHNICAL STACK DETAILS

LAYER	TECHNOLOGY	RESPONSIBILITY
Frontend Client	React 18, Styled-Components, Recharts	Role Dashboards, Real-time Charts, Responsive Layouts, Client-side routing, Export Utilities.
Java Backend	Spring Boot 3.2, Java 17/20, Hibernate JPA, Apache Maven	Business Logic, Multi-role REST Data APIs, Security Filter Chains, and Batch Scheduling.
Generative AI Integration	Google Gemini API (gemini-2.5-flash), Hugging Face Inference API	Intelligent Role-Specific Assistant (student performance, parent audits, attendance, and fee querying).
Database	MySQL 8.x	Relational Schemas, User Ledgers (single-table User model with polymorphic roles), Batches/Courses Catalog, Fee Logs.

SECURITY FILTER CHAIN & DECOUPLING MODEL

All network interactions use JSON-encoded requests. Spring Security handles endpoint protection, verifying JWT signature keys encoded in client-header HTTP bearer authentication tokens (Authorization: Bearer <accessToken>) on every dispatch through [SecurityConfig](#).

The backend employs database connection pools (HikariCP) to interact with relational schemas, ensuring fast transactional speeds during heavy academic logs updates, attendance submissions, and fee updates.

- Password storage:** Encrypted using BCrypt (strength 10) in the service layer before writing.
- JWT Claims mapping:** Custom filters map roles (ROLE_ADMIN, ROLE_TEACHER, ROLE_STUDENT, ROLE_PARENT, ROLE_ULTRA_SUPER_ADMIN) mapping controller security mappings.

2. ROLE HIERARCHIES & DASHBOARD PERMISSIONS

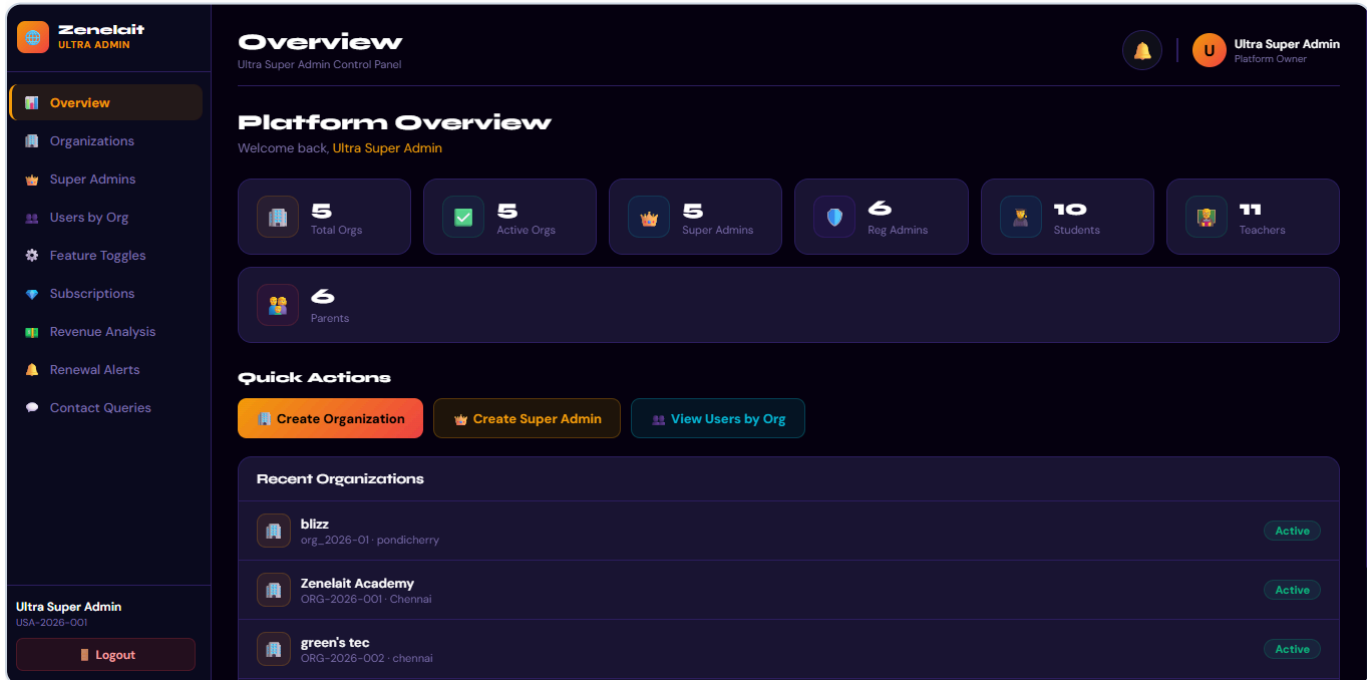
The platform enforces strict authentication boundaries across 6 distinct user roles:

ROLE	DATABASE FLAG MAPPING	OPERATIONAL CONTEXT PERMISSIONS
ULTRA SUPER ADMIN	ULTRA_SUPER_ADMIN	Full system owner. Manages organizations (tenants), global feature flags, subscription packages, and system-wide revenue forecasts.
SUPER ADMIN	ADMIN (superAdmin = true)	Academy owner. Manages organization-level admins, billing subscriptions, revenue reports, features, courses, and departments.
ADMIN	ADMIN (superAdmin = false)	Operational coordinator. Onboards teachers/students/parents, manages coursework catalogs, publishes announcements, logs fees.
TEACHER	TEACHER	Classroom executor. Publishes assignments, registers daily student attendance logs, evaluates and grades submissions.
STUDENT	STUDENT	Academic workspace. Submits assignments, requests course enrollments, prints verified course completion certificates.
PARENT	PARENT	Academic auditor. Links children using student codes, audits grades/attendance, manages parent wallet for Razorpay payments.

DATABASE POLYMORPHISM LAYOUT

Polymorphic role allocation is mapped using a single-table strategy in Hibernate (users table). User classes are loaded dynamically based on username/email credentials parsed by Spring Security filters.

3. ULTRA SUPER ADMIN DASHBOARD



OVERVIEW PAGE (PLATFORM DASHBOARD)

Loads stats via `GET /api/ultra-super-admin/stats` mapping counts:

- **Stats Cards:** Total Organizations, Active Organizations, Total Super Admins, Total Registered Admins, Total Students, Teachers, and Parents system-wide.
- **Quick Action buttons:**
 - *Create Organization:* Launches organization modal.
 - *Create Super Admin:* Launches super admin onboard modal.
 - *View Users by Org:* Switches tab to organization directory.

ORGANIZATIONS MANAGEMENT PAGE

Displays all onboarded academies in a detailed grid via `GET /api/ultra-super-admin/organizations`.

- **Columns:** Academy Name, Email, City, Organization Code, Active Plan, Active Status.
- **Create Button:** Launches creation dialog. Submits form inputs (Name, Email, Phone, Address, City, Country, Description) via `POST /api/ultra-super-admin/organizations`.
- **Status Toggle button (Disable/Enable):** Calls `PATCH /api/ultra-super-admin/organizations/{id}/toggle-active` to instantly disable or enable all login routes and APIs for that academy's users.

USERS BY ORG MANAGEMENT DIRECTORY

Permits the platform owner to inspect the operational directory inside any active tenant organization:

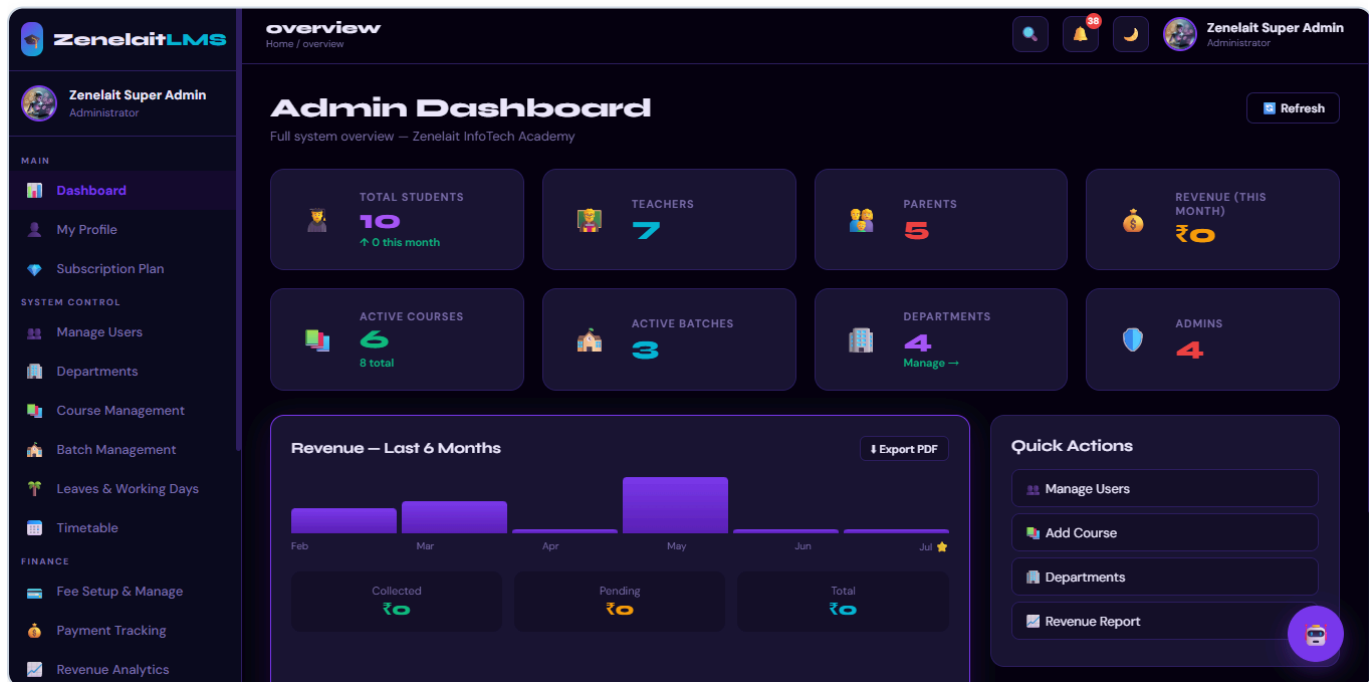
- **Data Collection:** Selecting an active organization triggers calls to:
 - `GET /api/ultra-super-admin/organizations/{id}/students`
 - `GET /api/ultra-super-admin/organizations/{id}/teachers`
 - `GET /api/ultra-super-admin/organizations/{id}/parents`
- **Information Display:** Summarizes counts for Students, Teachers, and Parents. Displays lists containing: User Name, System Code/ID, Email, Phone, and Active Status flag.

GRANULAR FEATURE MANAGEMENT & REVENUE COCKPIT

- **Global System Features:** Create and toggle platform modules globally using `POST /api/ultra-super-admin/features`.
- **Tenant Configuration:** Toggles specific global features for a selected organization via `POST /api/ultra-super-admin/organizations/{orgId}/features/{featureId}/toggle`.

- **Revenue Cockpit:** Fetches metrics via `GET /api/ultra-super-admin/revenue`. Displays MRR, ARR, and charts. Allows MoM target projections simulator slider adjustment.

4. SUPER ADMIN PORTAL (ACADEMY OWNER)



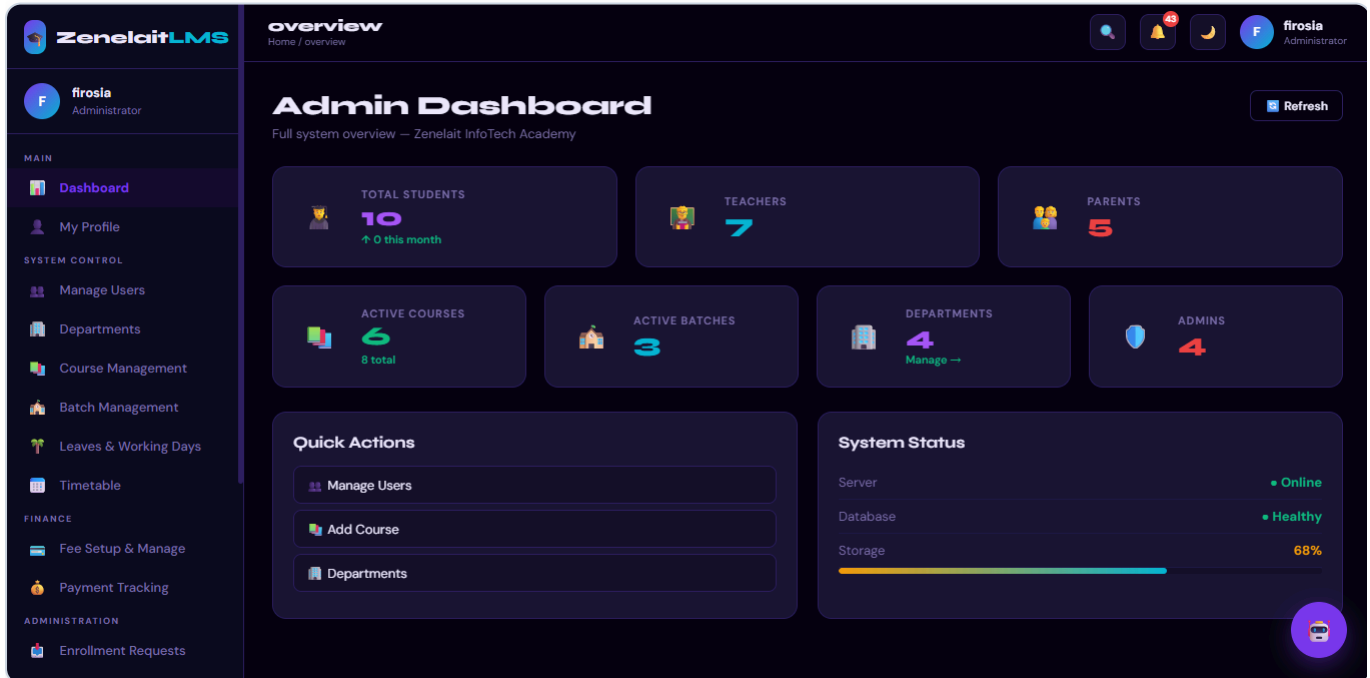
SIDEBAR PAGES & NAVIGATION MAPPING

- **Admin Management tab:** Accesses the local list of administrators. Featuring the "+ Add Admin" button, which registers regular administrators associated with their organization ID via `POST /api/admin/users`.
 - **Data Source:** Loads list of local administrators via `GET /api/admin/users?role=ADMIN`.
 - **Actions:** Delete button triggers `DELETE /api/admin/users/{id}` to permanently delete admin accounts.
- **Subscription Status page:** Views the active package details assigned by the Ultra Super Admin.
 - **Data Source:** Fetches active plan parameters via `GET /api/admin/subscription`, indicating package name, price, start date, and expiration date.
 - **Expiring warning:** Alerts the Super Admin on screen when the subscription is within 1 day of expiration. Includes a "Renew Plan Now" button which initiates subscription checkout via Razorpay payments.
- **Local Revenue Report page:** Generates dynamic graphs mapping monthly tuition fee collections.
 - **Data Source:** Pulls local transactional statistics via `GET /api/admin/revenue/summary`.
 - **Elements:** Displays total fees collected, pending balances, and overall invoicing volumes alongside historical bar charts.

SUPER ADMIN EXCLUSIVE CAPABILITIES

The Super Admin role acts as the security manager of the academy. Regular Admins are blocked from seeing the Admins list, Subscription plans, and Local Revenue Reports by route-guard conditionals (e.g. `!isSuperAdmin` blocks page execution).

5. ADMIN PORTAL (ACADEMIC OPERATIONS)



DASHBOARD PAGES & CONTROLS

- **Users Directories:** Tabbed tables showing Teachers, Students, and Parents.
 - **Data Source:** Fetch directories via:
 - `GET /api/admin/users?role=STUDENT`
 - `GET /api/admin/users?role=TEACHER`
 - `GET /api/admin/users?role=PARENT`
 - **Deactivate / Activate toggle buttons:** Calls `PATCH /api/admin/users/{id}/activate` to toggle active status flag.
 - **Delete button (🗑️):** Triggers `DELETE /api/admin/users/{id}` to permanently delete profiles.
 - **Add User button:** Launches onboard modal. Admin registers profiles specifying Role, Name, Email, Password, Department, and Phone, submitting data via `POST /api/admin/users`.
- **Course Catalog management page:** Lists academic subjects.
 - **Data Source:** Fetches courses via `GET /api/admin/courses`.
 - **Create Course button:** Launches modal. Submits title, department, duration hours, and teacherId via `POST /api/admin/courses`.
 - **Actions:** Update parameters via `PUT /api/admin/courses/{id}` or delete subjects via `DELETE /api/admin/courses/{id}`.
- **Batch Configurator page:** Groups students into active cohorts.
 - **Data Source:** Fetches cohorts via `GET /api/admin/batches`.
 - **Actions:**
 - Create batches specifying startDate, endDate, and department via `POST /api/admin/batches`.
 - Assign courses to batches via `POST /api/admin/batches/{id}/courses`.
 - Link students to batches via `POST /api/admin/batches/{id}/students`.

6. ADMIN PORTAL (TIMETABLES, FEES, AND SCHEDULING)

TIMETABLE SCHEDULER PAGE

Coordinates weekly lecture slots and classroom room numbers.

- **Data Source:** Loads active timetable slots via `GET /api/admin/timetable`.
- **Add Timetable Slot button:** Launches scheduler modal. Selects Day of Week, Start Time, End Time, Room Number, Course ID, and Batch ID, posting parameters via `POST /api/admin/timetable`.
- **Actions:** Delete timetable slots via `DELETE /api/admin/timetable/{slotId}`.

FEE INVOICING PAGE

Generates individual and cohort tuition fee invoices.

- **Data Source:** Fetches invoice records via `GET /api/admin/fees`.
- **Create Single Invoice:** Input studentId, amount, dueDate, and description. Submits billing record via `POST /api/admin/fees`.
- **Generate Batch-Wide Fee:** Billing modal allows administrators to select a specific batch and apply flat fees to all enrolled students at once via `POST /api/admin/fees/batch-generate`.
- **Action:** Mark as paid manually (reconciled off-platform) via `PATCH /api/admin/fees/{id}/mark-paid`.

ENROLLMENT REQUESTS PAGE

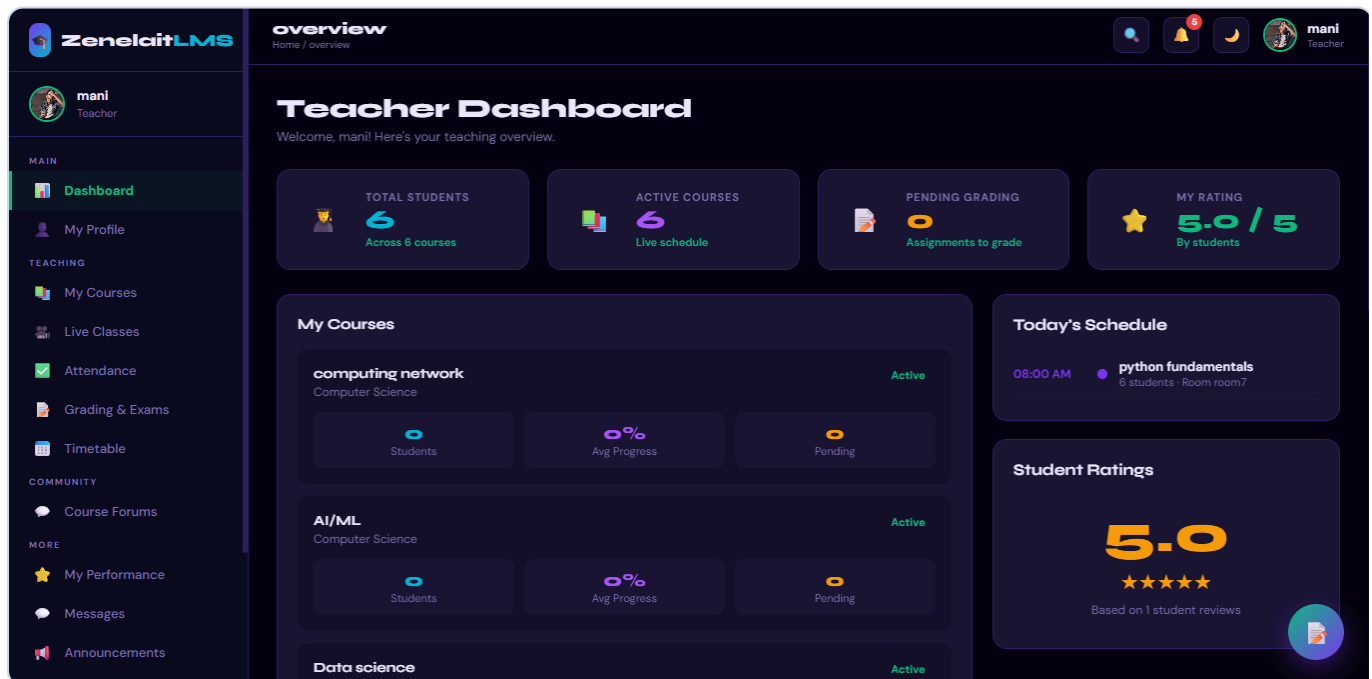
Handles incoming student registration requests.

- **Data Source:** Fetches active requests via `GET /api/admin/enrollments/requests`.
- **Approve button:** Approves enrollment requests, registering students into the course via `POST /api/admin/enrollments/requests/{id}/approve`.
- **Reject button:** Declines request, deleting it via `POST /api/admin/enrollments/requests/{id}/reject`.

MEETINGS & ANNOUNCEMENTS

- **Meetings Scheduler:** Admin schedules staff or classroom meetings (meeting link, topic, duration) via `POST /api/admin/meetings`.
- **Announcements:** Publishes notifications targeted by role via `POST /api/admin/announcements`.

7. TEACHER PORTAL (ACADEMIC CONTROLS)



OVERVIEW PAGE (TEACHER DASHBOARD)

Loads stats via `GET /api/teacher/stats` mapping counts:

- **Stats Cards:** Total Assigned Courses, Active Batches, Daily Lecture Hours, and Pending Grades.

MY COURSES VIEW

Fetches courses assigned to the teacher's profile via `GET /api/teacher/courses`.

ATTENDANCE REGISTRY PAGE

Logs student daily class presence. Selecting a course and date retrieves the student enrollment list:

- **Data Source:** Fetch enrollment lists via `GET /api/teacher/attendance?courseId={id}&date={date}`.
- **Log Attendance button:** Select status (PRESENT, ABSENT, LATE) for each student. Clicking "Save Attendance" submits the list via `POST /api/teacher/attendance`.

GRADING & REVIEW HUB

Manages assignments coursework and student evaluations.

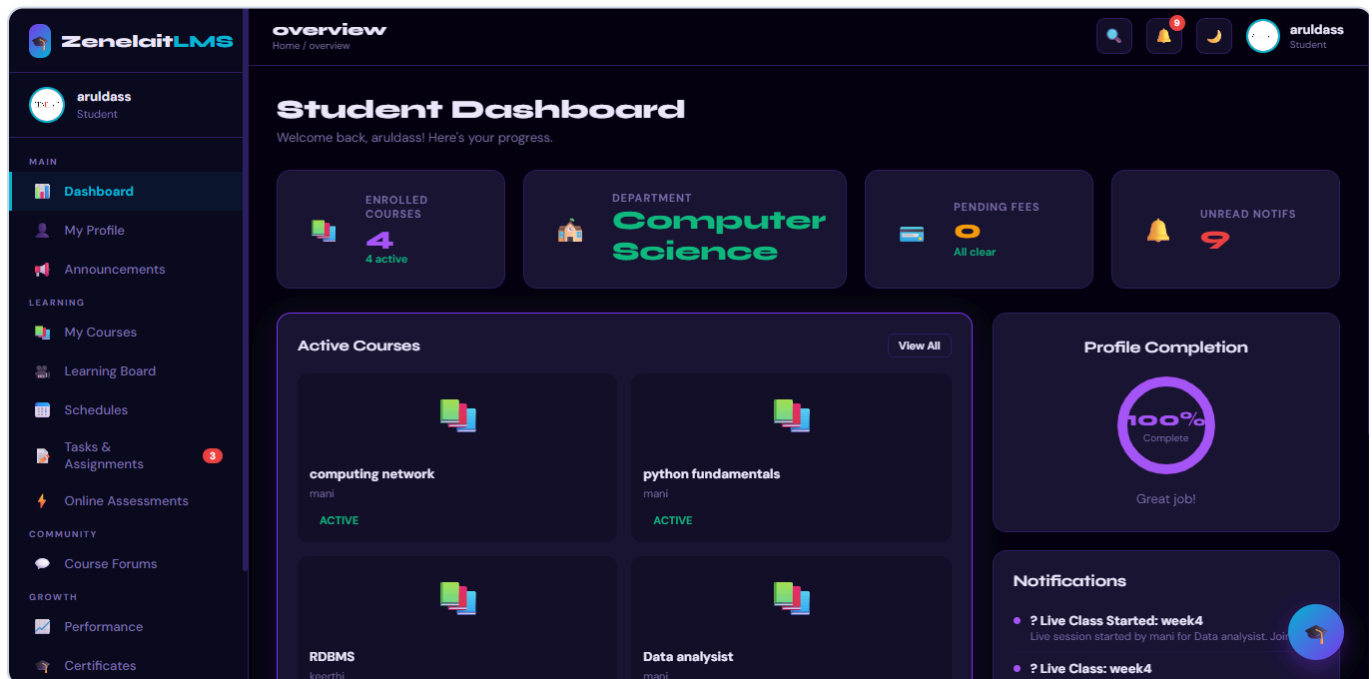
- **Create Assignment:** Add title, description, max marks, and due date. Publishes assignment to batch via `POST /api/teacher/assignments`.
- **Submissions Queue:** Lists student file uploads via `GET /api/teacher/submissions?courseId={id}`.
- **Grade submission:** Opens grading modal. Input marks and feedback comments. Submits results via `PATCH /api/teacher/submissions/{id}/grade`.

LIVE CLASSES MANAGEMENT

Launches real-time virtual classroom sessions.

- **Start Live Class button:** Creates Zoom or Google Meet links. Calls `POST /api/teacher/meetings/launch`. This updates the student dashboard in real-time, flashing an alert header to prompt students to join.
- **Close Meeting button:** Ends session, clearing alerts. Calls `PATCH /api/teacher/meetings/{id}/close`.

8. STUDENT PORTAL (LEARNING WORKSPACE)



COURSE CATALOG & ENROLLMENT REQUESTS

- **Data Source:** Loads available courses via `GET /api/student/courses/available`.
- **Request Enrollment button:** Submits registration request to the admin via `POST /api/student/courses/enroll?courseId={id}`.

SYLLABUS & COURSE TIMELINES PAGE

- **Data Source:** Loads lecture slides and class materials via `GET /api/student/courses/{id}/materials`.

SUBMISSIONS HUB

Uploads homework files and responses.

- **Data Source:** Fetch pending assignments via `GET /api/student/assignments`.
- **Submit Coursework button:** Opens upload modal. Attaches files or inputs responses. Submits assignments via `POST /api/student/assignments/{id}/submit`.

TIMED ASSESSMENTS & EXAM MONITOR

Integrates a secure browser-monitored quiz interface.

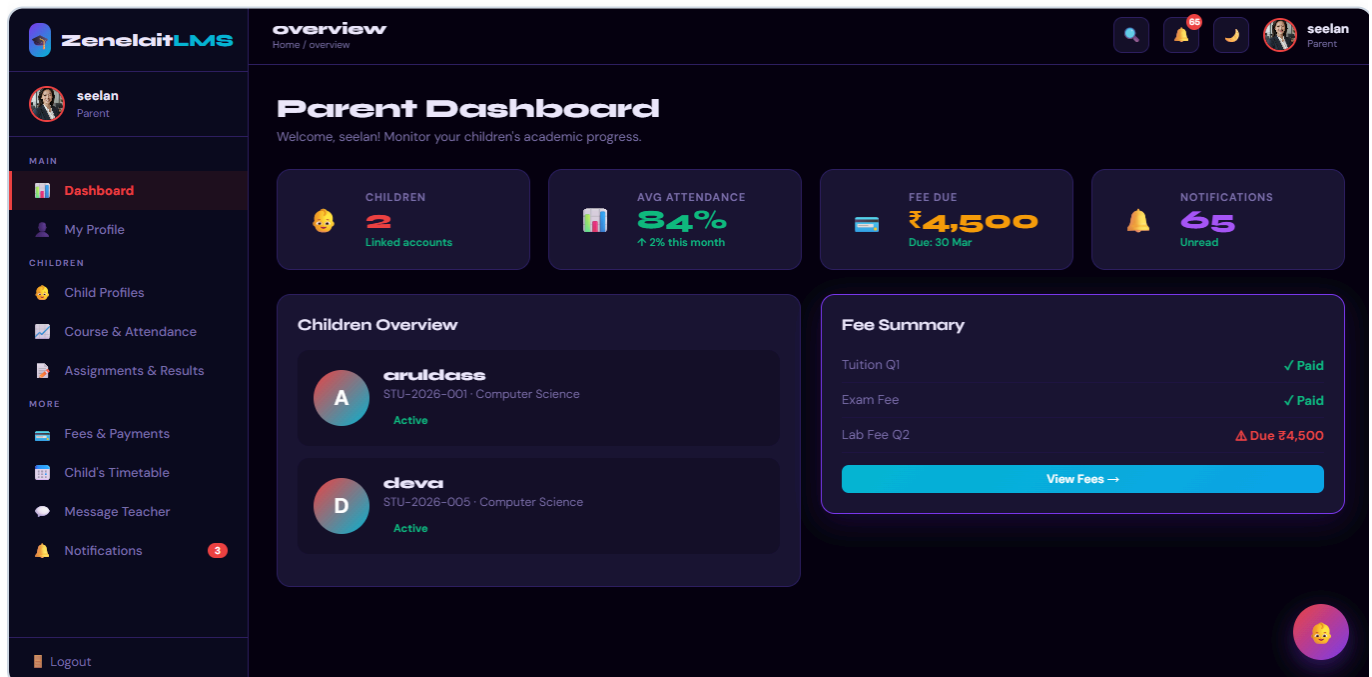
- **Start Assessment button:** Fetches questions and starts exam timer via `POST /api/student/assessments/{id}/start`.
- **Browser Blur Monitoring:** The system monitors tab switching. Tabbing away triggers a warning. On the 3rd warning, the exam automatically submits itself to prevent cheating via `POST /api/student/assessments/attempts/{id}/submit`.

COMPLETIONS & CERTIFICATES DRAWER

Prints verified course completion certificates.

- **Data Source:** Loads completed courses via `GET /api/student/certificates`.
- **Download Certificate button:** Renders completion details into a PDF document using `html2canvas` and `jspdf` libraries, verifying authenticity using database-registered completion hashes.

9. PARENT PORTAL (CHILD AUDITING & WALLET)



CHILDREN LINKING PAGE

Parents register their child profiles to activate academic audits.

- **Link Child button:** Opens link dialog. Input unique student code (e.g. 'STU-2026-001'). Submits linking request via `POST /api/parent/children/link`.

ACADEMIC AUDITS PAGE

Tracks children's grades, syllabus milestones, and attendance.

- **Attendance logs:** Gathers attendance histories via `GET /api/parent/children/{childId}/attendance`. Displays counts of present and absent classes alongside percentage statistics.
- **Grade sheets:** Gathers assignment grading records via `GET /api/parent/children/{childId}/grades`.

TUITION FEE WALLET & PAYMENTS

Integrated billing management.

- **Data Source:** Fetch outstanding tuition fees via `GET /api/parent/children/{childId}/fees`.
- **Top Up Wallet button:** Parents reload their wallet balance. Launches the Razorpay Payment Gateway modal. On success, wallet balances are updated via the Razorpay payment callback webhook.
- **Settle Invoice button:** Tapping "Settle Invoice" pays tuition bills. The system deducts the bill amount from the parent wallet ledger and marks the fee status as PAID.

MESSAGE CENTRE (PARENT-TEACHER THREAD)

Provides direct messaging channels with educators.

- **Search Teacher:** Searches teachers by name or department. Clicking "Start Chat" opens a chat window using `POST /api/parent/messages/conversations`.
- **Send Message button:** Posts text messages via `POST /api/parent/messages/send`.

10. AI ASSISTANT INTEGRATION (GEMINI PIPELINE)

Zenelait LMS features direct API integration with Google Gemini 2.5 Flash and Hugging Face Inference servers to provide contextual assistance based on active database records via `POST /api/ai/chat`.

PROMPT CONTEXT COLLECTION PIPELINE

When a user initiates a query, the backend executes context collection algorithms based on their role:

User Role	Database Mapped Context Gathers	LLM System Directive Prompt
Student	Class grades, assignment deadlines, attendance ratios, and course materials.	"Analyze this student's grade records and formulate personalized study suggestions and revision topics."
Parent	Linked children's performance, attendance statistics, and pending invoice dues.	"Review attendance rates and warn the parent of potential academic risks or upcoming due dates."
Teacher	Curriculum logs, course syllabus, student reviews, and submission marks.	"Assist the teacher in drafting evaluation criteria, template remarks, and syllabus goals."
Admin	Platform registrations, departments list, and active course catalogs.	"Provide administrative dashboard analytics and summarize overall enrollment metrics."

LLM API FORWARDING MECHANISM

The backend gathers contextual parameters, packages them alongside the user's message, and posts a payload to the Google Gemini API key configured in [application.properties](#). The response is returned to the chat box on the client dashboard.